RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world®

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Smart Building Energy Optimization Using Real-Time Streaming, Cybersecurity and Agentic AI

By

Project Report

Submitted by

STUDENT NAME 1 **USN 1RV23AIXXX**
STUDENT NAME 2 **USN 1RV23AIXXX**
STUDENT NAME 3 **USN 1RV23AIXXX**

in partial fulfilment for the requirement of 7th Semester

Stream Processing and Analytics (AI372IA)

Under the Guidance of

Dr.S.Anupama Kumar
Associate Professor
Department of AIML
RV College of Engineering
Bengaluru-59

Dr.Vijayalakshmi.M.N
Associate Professor
Department of AIML
RV College of Engineering
Bengaluru-59

Academic Year
2026 - 2027

RV COLLEGE OF ENGINEERING®

(Autonomous Institution Affiliated to Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Bengaluru– 560059

CERTIFICATE

This is to certify that the work entitled "Smart Building Energy Optimization Using Real-Time Streaming, Cybersecurity and Agentic AI" submitted in partial fulfillment of **Stream Processing and Analytics (AI372IA)** of VII Semester BE is a result of the bonafide work carried out by Student Name 1 (1RV23AIXXX), Student Name 2 (1RV23AIXXX) and Student Name 3 (1RV23AIXXX) during the Academic year 2026-27

Faculty In charge

Head of the Department

External Examination

Name of Examiners **Signature with date**

1.

2.

ABSTRACT

Commercial buildings consume a large share of electricity, and much of it is wasted by heating, cooling and lighting that do not follow how the building is actually used. Modern buildings stream sensor data continuously, which makes data-driven optimisation possible, but the same connected sensors also open the control loop to cyberattacks: a spoofed temperature or occupancy reading can make an optimiser waste energy or make rooms uncomfortable.

This project develops a real-time streaming platform that optimises the energy use of a simulated five-zone office building and protects that optimisation against manipulated sensor data. A physics-based simulator publishes readings to Apache Kafka; independent services validate and store them in PostgreSQL, compute energy and carbon emissions from live grid carbon intensity, predict each zone's power five minutes ahead with an XGBoost model, and choose comfortable, energy-efficient setpoints and light levels with a constrained SciPy optimiser. A security service checks every reading with physics rules and an Isolation Forest and every six-hour window with Kolmogorov–Smirnov and Population Stability Index drift tests.

Alerts are investigated by a LangGraph-based AI agent that re-checks the evidence, diagnoses the cause and proposes the smallest corrective action, such as quarantining a sensor and returning the zone to its safe schedule. Every proposal must pass a deterministic safety validator and, in manual mode, the explicit approval of a human operator, to whom each action is explained in plain language (optionally phrased by a local small language model). A Flask and Socket.IO dashboard shows the building, energy analytics, security monitoring, the agent's decisions, a live data-pipeline view and a chat assistant.

In a reproducible evaluation over three random seeds, the ML-based optimiser reduced energy use by 12.2 % and carbon emissions by 10.9 % compared with the baseline schedule while improving comfort compliance from 93.2 % to 94.5 %. All five attack types were detected, four of them within 15 simulated minutes, and every attack was contained and recovered from, retaining 98.1 % of the energy savings after recovery. The complete system passed an end-to-end verification of 15 checks in both control modes.

Table of Contents

Contents	Page No
Abstract	i
Table of Contents	ii
List of Tables	iii
List of Figures	iv
Introduction	1
1.1 Project Description	
1.2 Report Organization	
Problem Definition	2
2.1 Problem Statement	
2.2 Background Study	
Objectives	4
3.1 Primary Objectives	
Tools and Techniques Used	5
4.1 Tools	
4.2 Techniques	
Implementation	8
5.1 Code Snippets	
Results and Discussion	13
6.1 Final Results	
6.2 Discussion	
Conclusion	22
Future Enhancements	23
Appendix	24
Bibliography	25

LIST OF TABLES

Table I. Energy, Carbon, Peak Demand and Comfort over the Evaluation Period 13
Table II. Prediction Accuracy of the Energy Models on the Test Set 14
Table III. Attack Detection Performance (per Reading, after Warm-up) 15
Table IV. Containment and Recovery per Attack (Defended Scenario) 15
Table V. Kafka Transport Latency (Producer to Consumer) 16
Table VI. End-to-End System Verification (Both Control Modes) 20

LIST OF FIGURES

Figure 4.1. Architecture of the Smart Building Energy Optimization System 7

Figure 5.1. Configuration of Kafka, PostgreSQL, Control Mode, Agent and Approval Settings 8
Figure 5.2. Simulator Tick Producing One Reading per Zone 8
Figure 5.3. Publishing Sensor Readings to Apache Kafka 9
Figure 5.4. At-Least-Once Message Processing in the Kafka Consumer 9
Figure 5.5. Energy and Carbon Accounting for Each Reading 10
Figure 5.6. Online Energy Prediction Service Using the XGBoost Model 10
Figure 5.7. Population Stability Index and Classification of Suspicious Readings 11
Figure 5.8. LangGraph State Graph of the Incident Agent 11
Figure 5.9. Deterministic Safety Validation of Proposed Actions 12
Figure 5.10. Operator Decision Handling in the Approval Gate 12

Figure 6.1. Energy, Carbon and Peak Demand per Scenario 13
Figure 6.2. Energy Savings Compared with the Baseline Schedule 14
Figure 6.3. ML Prediction Accuracy per Scenario 14
Figure 6.4. Detection Metrics and Detection Latency per Attack 15
Figure 6.5. Processing Latency of the System Components 16
Figure 6.6. Live Building Overview Dashboard 16
Figure 6.7. Measured and Live Comparison of Baseline and Optimized Operation 17
Figure 6.8. Security Monitoring: Alerts per Hour and Simulated Attack Controls 17
Figure 6.9. Drift Monitoring with PSI Bands and Explanation 18
Figure 6.10. Agent Actions Waiting for Operator Approval with Plain-Language Explanations 18
Figure 6.11. Live Data Pipeline Visualisation 19
Figure 6.12. Chat Assistant Answering Questions from the Stored Data 20

Figure 9.1. Project Structure 24
Figure 9.2. Setup, Start-Up and Verification Commands (Windows) 24

CHAPTER-1
INTRODUCTION

This chapter introduces the project, Smart Building Energy Optimization Using Real-Time Streaming, Cybersecurity and Agentic AI, by presenting its motivation, scope and the main components of the system. It explains how real-time streaming, machine learning, optimisation, cyberattack detection and an AI agent with human oversight are combined into one working application, and outlines the organisation of the remaining chapters.

1.1 PROJECT DESCRIPTION

Buildings are responsible for a large part of global electricity use, and heating, ventilation and air conditioning (HVAC) and lighting account for most of it [7]. Fixed schedules heat, cool and light rooms whether or not people are present and regardless of the weather or the carbon intensity of the grid. Smart buildings stream sensor readings continuously, which allows energy use to be predicted and controlled in real time [8], [9].

The project builds such a system for a simulated office building with five zones: an open-plan office, private offices, a meeting room, a cafeteria and lobby, and a server room. A physics-based simulator produces temperature, humidity, CO_2, occupancy, lighting and power readings every simulated five minutes and publishes them to Apache Kafka. Independent services consume these readings to store them in PostgreSQL, compute energy and carbon emissions, predict each zone's next power value with an XGBoost model and choose energy-efficient setpoints and light levels with a constrained optimiser.

Because the optimiser trusts sensor data, manipulated readings can make it waste energy or harm comfort. A security service therefore checks every reading with physics rules and an Isolation Forest, and every window of readings with statistical drift tests. Alerts are investigated by an AI agent built with LangGraph, which re-checks the data, diagnoses the cause and proposes corrective actions. A deterministic safety validator approves or blocks every proposal and, in manual mode, a human operator must approve each action after reading a plain-language explanation.

A Flask dashboard with live Socket.IO updates presents the building, energy analytics, security monitoring, the agent's investigations and a live view of the data pipeline, and offers a chat assistant that answers questions from the stored data. The whole system runs on Windows or Linux with Kafka and PostgreSQL in Docker, and is evaluated with reproducible experiments and an automated end-to-end verification.

1.2 REPORT ORGANIZATION

The report is organised as follows. It begins with the Abstract, followed by the Table of Contents and the lists of tables and figures. Chapter 1 introduces the project. Chapter 2 defines the problem and reviews related work. Chapter 3 states the objectives. Chapter 4 describes the tools and techniques used. Chapter 5 explains the implementation with code snippets. Chapter 6 presents and discusses the experimental results and the working dashboard. Chapter 7 concludes the report, Chapter 8 discusses future enhancements, and the Appendix and Bibliography provide supporting material and references.

CHAPTER-2
PROBLEM DEFINITION

This chapter defines the problem addressed by the project and establishes its relevance for real buildings. It outlines the challenges of optimising energy use from live sensor data while protecting the control loop against manipulated readings, reviews related work on building energy prediction and control, stream processing, anomaly and drift detection and AI agents, and identifies the research gap that motivates the proposed system.

2.1 PROBLEM STATEMENT

Building energy management systems typically control HVAC and lighting with fixed schedules. They react neither to actual occupancy nor to the weather or to the carbon intensity of the electricity grid, and therefore waste energy and cause avoidable emissions. Data-driven optimisation can reduce this waste, but it depends on a continuous stream of trustworthy sensor data and on predictions that are accurate enough to compare control options.

At the same time, connected building sensors are part of a cyber-physical system that can be attacked [11]. An attacker who spoofs temperature readings, hides occupancy, falsifies power-meter values, replays old data or slowly biases readings can make an optimiser take harmful decisions while every individual value still looks plausible. Detecting such attacks in real time is difficult, and responding automatically is risky: an automated response that is itself wrong can disrupt the building.

Therefore, there is a need for an integrated, real-time system that streams and processes building data, predicts and optimises energy use with comfort constraints, detects both sudden and gradual manipulation of sensor data, and responds to confirmed threats with explainable, safety-checked actions that a human operator can approve before anything is changed.

2.2 BACKGROUND STUDY

Building energy consumption has been studied extensively. Pérez-Lombard et al. showed that buildings account for a large and growing share of energy use, with HVAC as the dominant end use [7]. Amasyali and El-Gohary reviewed data-driven prediction of building energy consumption and found that machine-learning models such as tree ensembles and neural networks outperform simple statistical methods when sufficient sensor history is available [8]. Candanedo et al. published a widely used dataset of appliance energy use in a low-energy house and compared regression models for predicting it [14]; this dataset is also used in this project for additional experiments. Chen and Guestrin introduced XGBoost, a scalable gradient-boosted tree method that is widely used for tabular prediction tasks of this kind [2].

For control, Drgoňa et al. surveyed model predictive control for buildings and showed that optimisation with forecasts can save energy while keeping comfort constraints, but also that it depends on accurate models and reliable measurements [9]. Wang and Hong reviewed reinforcement learning for building control and identified data requirements, safety and interpretability as the main obstacles to deployment [10]. This project uses a constrained sequential quadratic programming method (SLSQP) [21] from SciPy [15] with ML forecasts, which keeps every decision within explicit comfort and safety limits.

Real-time data pipelines commonly use Apache Kafka, a distributed, partitioned log designed for high-throughput messaging between independent producers and consumers [1]. Kafka allows each service to process the stream at its own pace and to recover after a failure without

losing messages, which is essential when storage, prediction, optimisation and security analysis run as separate services.

Security of cyber-physical systems has been surveyed by Humayed et al., who highlight false-data injection and replay as key threats to sensing and control [11]. Unsupervised anomaly detection is attractive when labelled attacks are rare: Liu et al. proposed the Isolation Forest, which isolates anomalies with random partitions [3], and Mirsky et al. showed that online ensembles can detect intrusions in streaming data [12]. Slow manipulation, however, is better treated as distribution change: Gama et al. surveyed concept-drift detection [4], the Kolmogorov–Smirnov test compares two samples without distributional assumptions [5], and the Population Stability Index measures how much a distribution has shifted [6].

Large language models can act as agents that reason, call tools and act, as shown by the ReAct framework [13]. LangGraph [17] provides stateful agent workflows in which tool calls and decisions form an explicit graph. Using such agents for operational decisions raises questions of safety and trust; guidelines for human–AI interaction stress that people must be able to understand, correct and override AI behaviour [22].

Research Gap: Existing studies address energy prediction, building control, stream processing and anomaly detection largely in isolation. Few systems combine real-time streaming, ML-based energy optimisation, detection of both sudden and gradual sensor manipulation, and an explainable AI agent whose actions are checked by deterministic safety rules and approved by a human operator, and evaluate the effect of attacks and defences on energy savings in one reproducible setting. This project addresses that gap.

CHAPTER-3
OBJECTIVES

This chapter presents the primary objectives of the project and describes the goals that the system is designed to achieve. It highlights the intended outcomes in streaming data processing, energy prediction and optimisation, cyberattack detection, explainable and safe corrective action, and the evaluation and visualisation of the complete system.

3.1 PRIMARY OBJECTIVES

1. To build a real-time streaming pipeline in which a simulated smart building publishes sensor readings to Apache Kafka and independent services validate, process and store them in PostgreSQL.
2. To predict each zone's energy consumption with machine learning and to optimise HVAC setpoints and lighting levels for lower energy use and carbon emissions without reducing occupant comfort.
3. To detect simulated cyberattacks on building sensors, including temperature spoofing, occupancy manipulation, abnormal power, replay and gradual manipulation, and to separate them from ordinary data drift.
4. To develop an AI agent that investigates alerts, explains its diagnosis and proposes corrective actions that are validated by a deterministic safety layer and approved by a human operator.
5. To provide a live web dashboard that visualises sensor data, energy and carbon, predictions, security alerts, agent decisions and the data pipeline, and answers questions through a chat assistant.
6. To evaluate the system reproducibly in terms of energy savings, carbon reduction, peak demand, prediction accuracy, detection accuracy, recovery time and processing latency.

CHAPTER-4
TOOLS AND TECHNIQUES USED

This chapter describes the tools, technologies and techniques employed in the development of the system. It provides an overview of the software frameworks, libraries and platforms used, along with their roles in the architecture, and explains the techniques adopted for streaming, energy prediction and optimisation, attack and drift detection, agentic decision making and visualisation.

4.1 TOOLS USED

- **Python 3:** the main language for the simulator, all streaming services, the ML models, the agent and the dashboard.
- **Apache Kafka:** the event-streaming platform that carries readings, predictions, commands, alerts, agent actions and operator decisions between services through nine topics (Docker, KRaft mode).
- **confluent-kafka:** the Python Kafka client used for producers and consumer groups with at-least-once delivery.
- **PostgreSQL:** the relational database that stores all readings, energy records, predictions, alerts, actions and agent decisions (Docker); psycopg 3 is used as the driver.
- **XGBoost and scikit-learn:** XGBoost predicts power; scikit-learn provides the baseline models, the Isolation Forest and evaluation metrics.
- **SciPy:** the SLSQP optimiser for setpoints and lighting and the Kolmogorov–Smirnov test for drift detection.
- **NumPy and Pandas:** numerical processing, feature engineering and analysis of experiment results.
- **LangGraph and Anthropic SDK:** LangGraph builds the agent's state graph; the Anthropic SDK connects it to Claude when an API key is configured, otherwise an offline rule-based policy is used.
- **Ollama (optional):** runs a small language model locally to phrase explanations and chat answers.
- **Flask, Flask-SocketIO, Bootstrap and Plotly.js:** the web dashboard with live push updates and interactive charts.
- **Open-Meteo and Electricity Maps:** external APIs for outdoor weather and grid carbon intensity.
- **Docker Desktop, Matplotlib and pytest:** containers for Kafka and PostgreSQL, figures for the evaluation report, and 434 automated tests.

4.2 TECHNIQUES USED

1. Physics-based building simulation

Each zone is modelled with thermal mass, heat exchange with the outdoors, internal gains from people and equipment, CO_2 from occupants, and HVAC and lighting power. Seeded randomness makes runs reproducible, and simulated attacks manipulate the published readings without changing the true building state.

2. Event-driven streaming architecture

Every service has its own Kafka consumer group, so each receives every message of its topics. Offsets are stored only after a message has been processed, and every database write is idempotent, which gives at-least-once processing without duplicates.

3. Feature engineering and energy accounting

Power is integrated into energy, split into HVAC, lighting and appliance shares, and multiplied by the grid carbon intensity to obtain CO_2 emissions. Rolling statistics, time-of-day features and lags are computed for the ML model.

4. Energy prediction

An XGBoost regressor predicts each zone's average power for the next five-minute interval from 37 features. It is trained on 28 simulated days with a chronological train/validation/test split and compared with persistence, linear regression and random forest baselines.

5. Constrained optimisation

Every 15 simulated minutes per zone, SLSQP minimises a weighted objective of predicted energy, carbon and discomfort subject to absolute setpoint limits, comfort bands, a maximum step and minimum lighting for occupied zones.

6. Attack and anomaly detection

Physics rules check consistency (for example, CO_2 must rise when people are present), and a per-zone Isolation Forest flags readings unlike normal data. Each alert is classified as one of the attack types or as an unknown anomaly.

7. Drift detection

Six-hour windows are compared with the training data using the KS test and PSI. Security drift monitors residuals that should never change, which indicates gradual manipulation; data drift monitors ordinary inputs, which indicates that the model needs retraining but not an attack.

8. Agentic investigation with tools

A LangGraph state graph gives the agent tools to read sensor and energy data, check weather and carbon, re-run the detectors and drift analysis, and propose actions. It submits a structured assessment with cause, confidence and impact.

9. Deterministic safety validation and human approval

The safety validator checks every proposal against fixed rules: limits, comfort and equipment bands, data freshness, evidence for sensor isolation and rate limits. In manual mode, approved actions wait for the operator and are validated again before they run.

10. Live visualisation

A background task polls PostgreSQL and pushes new readings, alerts, decisions and actions to all browsers over Socket.IO; the dashboard never consumes Kafka directly.

The overall architecture of the system is shown in Figure 4.1. Data flows from the building simulator and external APIs through Kafka to the processing services, is stored in PostgreSQL and shown on the dashboard, while approved actions return to the building through the safety validator and the operator.

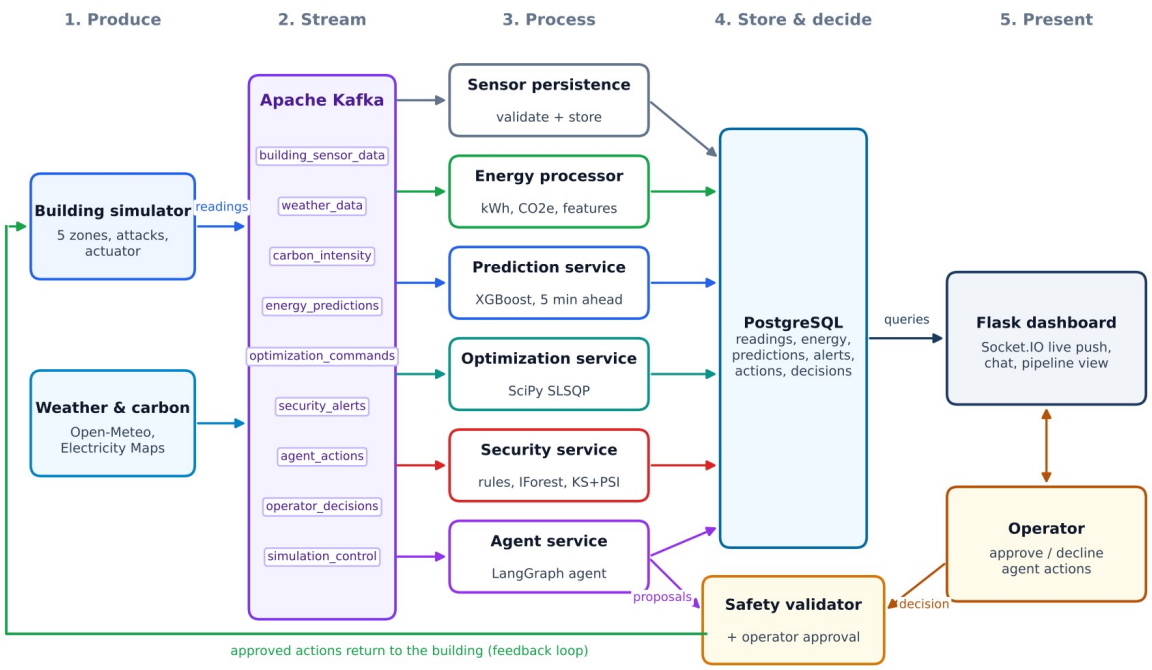


Figure 4.1. Architecture of the Smart Building Energy Optimization System

CHAPTER-5
IMPLEMENTATION

This chapter presents the key implementation components of the system. It highlights the code responsible for configuration, simulation, Kafka-based streaming, energy processing, prediction, attack and drift detection, the LangGraph agent, the safety validator and the operator approval gate. The selected snippets illustrate how these components work together to form a real-time, secure and explainable energy optimisation pipeline.

5.1 CODE SNIPPETS

```
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
POSTGRES_HOST=localhost
POSTGRES_DB=smart_building
SIM_INTERVAL_SECONDS=2
SIM_SPEED_FACTOR=1
WEATHER_PROVIDER=open-meteo
CARBON_PROVIDER=electricitymaps
AGENT_LLM_PROVIDER=auto
AGENT_MIN_ALERT_SEVERITY=medium
AGENT_APPROVAL_MODE=manual
AGENT_APPROVAL_TIMEOUT_S=300
CONTROL_MODE=advisory
SECURITY_DETECTORS=auto
DASHBOARD_POLL_INTERVAL_S=2
SLM_PROVIDER=off
SLM_MODEL=qwen2.5:1.5b
```

Figure 5.1. Configuration of Kafka, PostgreSQL, Control Mode, Agent and Approval Settings

Figure 5.1 shows the central configuration in the .env file. It defines the Kafka broker and the PostgreSQL database, the simulation interval and speed, the weather and carbon providers, the control mode, the agent's minimum alert severity, the operator approval mode with its timeout and the optional small language model. All settings are validated at start-up by config.py.

```
    def tick(self) -> list[dict[str, Any]]:
        """Advance one reading interval and return one event per zone."""
        interval = self.settings.simulated_seconds_per_tick
        self._advance(interval)
        self.ticks += 1
        emitted_at = datetime.now(timezone.utc).isoformat(timespec="milliseconds")
        outdoor_readings = self.sensors.read_outdoor(self.building.outdoor)
        events = []
        for zone_id, zone in self.building.zones.items():
            self._sequence[zone_id] += 1
            event = {
                "event_id": str(uuid.UUID(int=self._event_rng.getrandbits(128), version=4)),
                "event_type": EVENT_TYPE,
                "schema_version": SCHEMA_VERSION,
                "source": "simulator",
                "building_id": self.settings.building_id,
                "zone_id": zone_id,
                "zone_name": zone.spec.name,
                "zone_type": zone.spec.zone_type,
                "sequence": self._sequence[zone_id],
                "timestamp": self.sim_time.isoformat(timespec="seconds"),
                "emitted_at": emitted_at,
                "interval_seconds": round(interval, 3),
                **self.sensors.read_zone(zone),
                **outdoor_readings,
                "energy_kwh": round(zone.state.energy_kwh, 5),
```

Figure 5.2. Simulator Tick Producing One Reading per Zone

The simulator advances the building physics for one interval and creates one sensor event per zone, as shown in Figure 5.2. Each event carries an identifier that depends on the random seed and the start time, a sequence number and all measurements; active simulated attacks manipulate the event before it is published.

```
producer = JsonProducer(config.KAFKA, name=SERVICE)
simulator = BuildingSimulator(settings)

def publish(event: dict[str, Any]) -> None:
    producer.send(BUILDING_SENSOR_DATA, event, key=event["zone_id"], headers={"event_type": event["event_type"]})
```

Figure 5.3. Publishing Sensor Readings to Apache Kafka

Figure 5.3 shows how every reading is published to the building_sensor_data topic with the zone identifier as the message key. Keying by zone keeps the readings of one zone in order within a partition, which the stateful services rely on.

```
        attempts, transient_attempt = self.max_handler_retries + 1, 0
        attempt = 0
        while True:
            try:
                handler(consumed)
                self.stats.processed += 1
                break
            except TransientProcessingError as exc:
                if stop_event.is_set():
                    logger.warning("Stopping during transient failure; message will be "
                                   "processed again after restart", extra=location)
                    return  # offset not stored
                transient_attempt += 1
                self.stats.transient_retries += 1
                delay = min(self.MAX_TRANSIENT_BACKOFF_S,
                            self.retry_backoff_s * 2 ** min(transient_attempt - 1, 10))
                logger.warning("Transient processing failure; retrying", extra={
                    **location, "attempt": transient_attempt, "retry_in_s": round(delay, 2),
                    "error": str(exc)})
                stop_event.wait(delay)
        ...
        self._commit(message)   # offset stored only after the message was handled
```

Figure 5.4. At-Least-Once Message Processing in the Kafka Consumer

The shared consumer in Figure 5.4 retries transient failures (for example, a database outage) with exponential back-off and stores a message's offset only after it has been processed. If a service stops, unprocessed messages are delivered again after restart, and idempotent writes prevent duplicates.

```python
    def process(self, reading: SensorReadingRecord) -> EnergyRecord:
        total = reading.total_power_kw
        energy = reading.energy_kwh
        if total > 0:
            hvac_e = energy * reading.hvac_power_kw / total
            lighting_e = energy * reading.lighting_power_kw / total
        else:
            hvac_e = lighting_e = 0.0
        appliance_e = max(0.0, energy - hvac_e - lighting_e)

        intensity = self.carbon_intensity_g_per_kwh
        return EnergyRecord(
            event_id=reading.event_id,
            building_id=reading.building_id,
            zone_id=reading.zone_id,
            event_time=reading.event_time,
            interval_seconds=reading.interval_seconds,
            energy_kwh=energy,
            hvac_energy_kwh=hvac_e,
            lighting_energy_kwh=lighting_e,
            appliance_energy_kwh=appliance_e,
            total_power_kw=total,
            carbon_intensity_g_per_kwh=intensity,
            carbon_kg_co2e=energy * intensity / 1000 if intensity is not None else None,
```

Figure 5.5. Energy and Carbon Accounting for Each Reading

The energy processor in Figure 5.5 splits each reading's energy into HVAC, lighting and appliance shares in proportion to their power and converts energy into CO_2 emissions with the latest grid carbon intensity received from the carbon_intensity topic.

```python
    def __call__(self, message: ConsumedMessage) -> None:
        try:
            reading = validate_sensor_event(message.value)
        except ValidationError as exc:
            self.stats["invalid"] += 1
            self.logger.debug("Invalid event skipped", extra={"errors": exc.errors})
            return
        row = {column: getattr(reading, column) for column in BUILDING_COLUMNS}
        row["event_id"] = str(reading.event_id)
        self._check_interval(reading.interval_seconds)

        prediction = self.online.update(row)
        if prediction is None:
            self.stats["no_prediction"] += 1   # warming up, duplicate or gap
            return
        horizon_s = reading.interval_seconds * self.bundle.spec.horizon
        predicted_for = reading.event_time + timedelta(seconds=horizon_s)
        self.producer.send(ENERGY_PREDICTIONS, {
            "prediction_id": str(uuid.uuid5(_NAMESPACE, f"{reading.event_id}|{self.bundle.version}")),
            "source_event_id": str(reading.event_id),
            "building_id": reading.building_id,
            "zone_id": reading.zone_id,
            "event_time": reading.event_time.isoformat(),
            "predicted_for": predicted_for.isoformat(),
            "horizon_seconds": horizon_s,
            "predicted_power_kw": round(prediction.value, 4),
            "predicted_energy_kwh": round(prediction.value * horizon_s / 3600, 6),
            "model_name": self.bundle.model_name,
            "model_version": self.bundle.version,
            "created_at": datetime.now(timezone.utc).isoformat(timespec="milliseconds"),
```

Figure 5.6. Online Energy Prediction Service Using the XGBoost Model

Figure 5.6 shows the prediction service. It validates each reading, updates the online feature window and, once enough history is available, predicts the next interval's power. The prediction is published to the energy_predictions topic with the model version, so that results remain traceable to the trained model.

```python
def population_stability_index(reference: np.ndarray, current: np.ndarray, bins: int = PSI_BINS) -> float:
    """PSI with quantile bins of the reference distribution."""
    edges = np.unique(np.quantile(reference, np.linspace(0, 1, bins + 1)))
    if len(edges) < 3:                      # (almost) constant reference
        edges = np.array([reference.min() - 1e-9, reference.max() + 1e-9])
    edges[0], edges[-1] = -np.inf, np.inf
    ref_share = np.histogram(reference, edges)[0] / len(reference)
    cur_share = np.histogram(current, edges)[0] / len(current)
    ref_share = np.clip(ref_share, PSI_EPSILON, None)
    cur_share = np.clip(cur_share, PSI_EPSILON, None)
    return float(np.sum((cur_share - ref_share) * np.log(cur_share / ref_share)))


def classify(hits: list[RuleHit], anomaly: tuple[bool, float, float] | None,
             top_group: tuple[str, str, float] | None) -> tuple[str | None, str]:
    """Return (attack_type or None, detector) for one reading."""
    is_anomaly = bool(anomaly and anomaly[0])
    if hits:
        votes: dict[str, int] = defaultdict(int)
        for hit in hits:
            votes[hit.attack_type] += 1
        # Replay evidence is the most specific; otherwise the most frequent type.
        attack = "replay" if "replay" in votes else max(votes, key=lambda k: (votes[k], k))
        return attack, "rules+isolation_forest" if is_anomaly else "rules"
    if is_anomaly:
        if top_group and top_group[2] >= MIN_GROUP_Z:
            return GROUP_TO_ATTACK[top_group[0]], "isolation_forest"
        return "unknown_anomaly", "isolation_forest"
    return None, ""
```

Figure 5.7. Population Stability Index and Classification of Suspicious Readings

Figure 5.7 contains two detection functions. The PSI compares the distribution of a recent window with the reference distribution using quantile bins. The classification function combines the votes of the physics rules with the Isolation Forest result to label a reading as a specific attack type or an unknown anomaly.

```python
    def _build(self):
        graph = StateGraph(AgentState)
        graph.add_node("context", self._context)
        graph.add_node("agent", self._agent)
        graph.add_node("tools", self._tools)
        graph.add_node("finalize", self._finalize)
        graph.add_node("safety_gate", self._safety_gate)
        graph.add_node("execute", self._execute)
        graph.add_node("record", self._record)
        graph.add_edge(START, "context")
        graph.add_edge("context", "agent")
        graph.add_conditional_edges("agent", self._route, {"tools": "tools", "finalize": "finalize"})
        graph.add_edge("tools", "agent")
        graph.add_edge("finalize", "safety_gate")
        graph.add_edge("safety_gate", "execute")
        graph.add_edge("execute", "record")
        graph.add_edge("record", END)
        return graph.compile()
```

Figure 5.8. LangGraph State Graph of the Incident Agent

The incident agent is a LangGraph state graph, as shown in Figure 5.8. It gathers context, alternates between the language model (or offline policy) and tool calls, finalises an assessment, passes every proposal through the safety gate, executes approved actions and records the decision.

```
    def validate(self, action: ProposedAction, *, latest_event: dict[str, Any] | None, now: datetime,
                 zone_alerts: list[dict[str, Any]]) -> SafetyVerdict:
        problems = self._structure(action)
        if problems:
            return SafetyVerdict(False, tuple(problems))
        if action.action_type in ("isolate_sensor", "restore_sensor"):
            problems = self._sensor_action(action, zone_alerts, now)
        elif action.action_type == "clear_overrides":
            problems = []
        else:
            problems = self._control_action(action, latest_event, now, zone_alerts)
        # The action itself does not count (it was recorded when it first passed, before operator approval).
        if not problems and self._recent_count(action.zone_id, now, exclude=action.action_id) \
                >= self.agent.max_actions_per_zone_per_hour:
            problems.append(f"rate limit: already {self.agent.max_actions_per_zone_per_hour} approved "
                            f"actions for {action.zone_id} in the last hour")
        if problems:
            return SafetyVerdict(False, tuple(problems))
        return SafetyVerdict(True, (self._approval_text(action),))

    def _structure(self, action: ProposedAction) -> list[str]:
```

Figure 5.9. Deterministic Safety Validation of Proposed Actions

The safety validator in Figure 5.9 is deterministic: the same proposal in the same state always receives the same verdict. It checks the structure of the action, applies rules for sensor isolation and restoration, checks control actions against the optimiser's comfort and equipment constraints, and enforces a rate limit per zone.

```
    def decide(self, action_id: str, decision: str, operator: str = "operator",
               note: str | None = None) -> ProposedAction | None:
        """Apply an operator decision; returns the action, or ``None`` if it is not waiting (any more)."""
        with self._lock:
            entry = self._pending.pop(str(action_id), None)
        if entry is None:
            return None
        action, now = entry.action, self.data.now() or datetime.now(timezone.utc)
        comment = f" ({note})" if note else ""
        # Kept with the explanation, so it survives later updates of the result (e.g. the simulator's).
        action.explanation = {**(action.explanation or {}), "decision": {"decision": decision, "by": operator,
                                                                         "note": note, "at_sim_time": now.isoformat()}}
        if decision == "decline":
            action.status, action.result = "declined", f"declined by {operator}{comment}; nothing was changed"
            self.stats["declined"] += 1
        else:
            verdict = self.validator.validate(action, latest_event=self.data.latest_event(action.zone_id), now=now,
                                              zone_alerts=self.data.alerts(action.zone_id, hours=24))
            if not verdict.approved:
                action.status = "rejected"
                action.safety_reason = f"re-checked when approved: {verdict.text}"
                action.result = f"approved by {operator}{comment}, but the situation changed and the safety re-check failed"
                self.stats["recheck_failed"] += 1
            else:
                status, result = self.gateway.execute(action, now)
                action.status, action.result = status, f"approved by {operator}{comment}; {result}"
                self.stats["approved"] += 1
        self.recorder.record_actions([action])
        logger.info("Operator decision applied", extra={"action_id": action.action_id, "decision": decision,
                                                        "status": action.status})
        return action
```

Figure 5.10. Operator Decision Handling in the Approval Gate

Figure 5.10 shows how an operator's decision is applied. A declined action changes nothing. An approved action is validated again against the latest state, because the situation may have changed while it was waiting, and only then executed. Every outcome is recorded in PostgreSQL and on Kafka.

CHAPTER-6
RESULTS AND DISCUSSION

This chapter presents the results obtained by implementing and testing the system. It includes the experimental setup, measurements of energy, carbon, peak demand and comfort, prediction accuracy, attack detection, corrective action and recovery, processing latency and an end-to-end system verification, together with screenshots of the working dashboard. The discussion evaluates whether the objectives were achieved and explains the limitations.

6.1 FINAL RESULTS

Experimental Setup

All results were computed from raw experiment files; nothing was entered by hand. The main evaluation simulated four days from 9 February 2026 with five-minute readings for five zones, repeated for three random seeds (101, 202 and 303) that change occupancy, weather and sensor noise. The first 24 hours were an attack-free warm-up. Four scenarios were compared for each seed: (1) the baseline schedule, (2) the ML-based optimiser, (3) the optimiser under five attacks without defence, and (4) the optimiser under the same attacks with live detection, the agent (offline rule-based policy), the safety validator, sensor isolation and automatic recovery. The carbon intensity profile was synthetic and identical in all scenarios. Values are reported as mean ± standard deviation over the seeds.

Table I. Energy, Carbon, Peak Demand and Comfort over the Evaluation Period

Scenario	Energy (kWh)	Saving	CO_2e (kg)	Peak (kW)	Comfort
1 Baseline	1806.4 ± 29.3	0.0 %	527.3 ± 7.9	38.4 ± 0.1	93.2 %
2 ML-optimized	1586.5 ± 6.4	12.2 %	469.6 ± 1.9	35.9 ± 1.1	94.5 %
3 Attack, no defence	1601.3 ± 4.9	11.3 %	473.7 ± 1.6	35.2 ± 0.5	94.3 %
4 Attack + detection + agent	1637.1 ± 14.9	9.4 %	482.4 ± 4.1	36.3 ± 2.4	94.0 %

Table I shows that the ML-based optimiser reduced energy use by 12.2 % and CO_2 emissions by 10.9 % compared with the baseline schedule, lowered peak demand by 6.4 % and improved comfort compliance (occupied time within 20–25 °C) from 93.2 % to 94.5 %. Under attack the defended system still saved 9.4 % of energy; the difference to scenario 2 comes mainly from zones running on their safe baseline schedule while a sensor was quarantined.

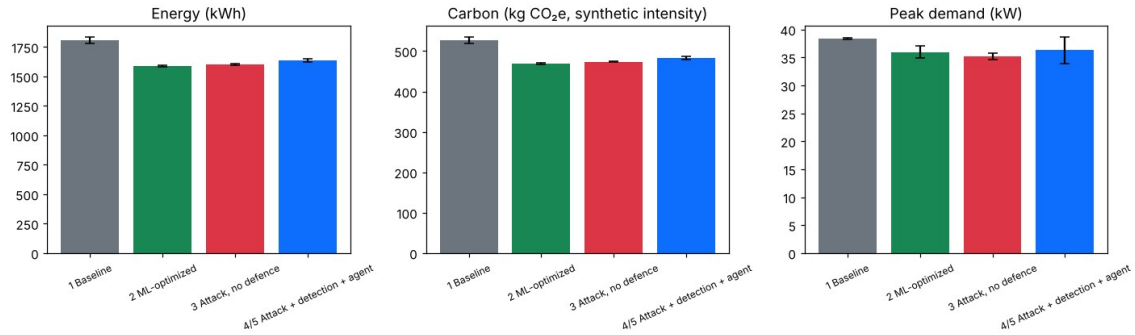


Figure 6.1. Energy, Carbon and Peak Demand per Scenario

Figure 6.1 compares the four scenarios across energy, carbon and peak demand. The savings of the optimised scenarios are consistent across seeds, as shown by the small error bars.

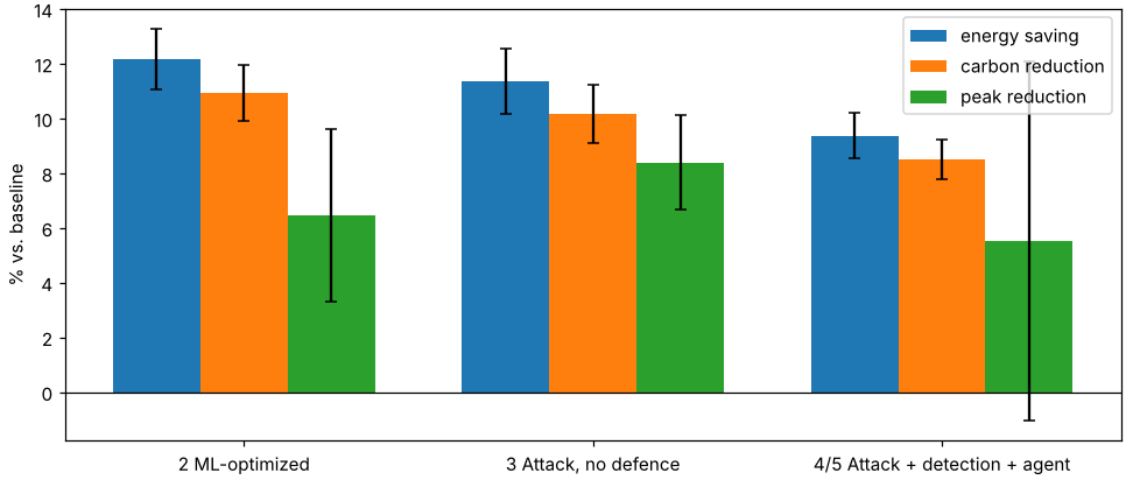


Figure 6.2. Energy Savings Compared with the Baseline Schedule

Figure 6.2 shows the savings relative to the baseline for the whole period, the attack window and the post-recovery window. After all sensors had been restored, the defended system retained 98.1 ± 4.2 % of the savings of the undisturbed optimiser. A separate week-long comparison measured savings of 7.4 % in a winter week and 12.6 % in a summer week, with comfort compliance improving from 88.7 % to 93.1 % and from 85.7 % to 95.7 % respectively.

Table II. Prediction Accuracy of the Energy Models on the Test Set

Model	**MAE (kW)**	**RMSE (kW)**	**R^2**
Persistence (naive)	0.158	0.359	0.980
Linear regression	0.169	0.346	0.981
Random forest	0.112	0.220	0.992
XGBoost (deployed)	0.116	0.208	0.993

Table II lists the accuracy of the next-interval power models on 6,040 test readings. XGBoost and random forest clearly outperform the persistence and linear baselines; XGBoost has the lowest RMSE and R^2 of 0.993, while random forest has a slightly lower MAE. XGBoost was deployed for its lower large-error rate and smaller model size.

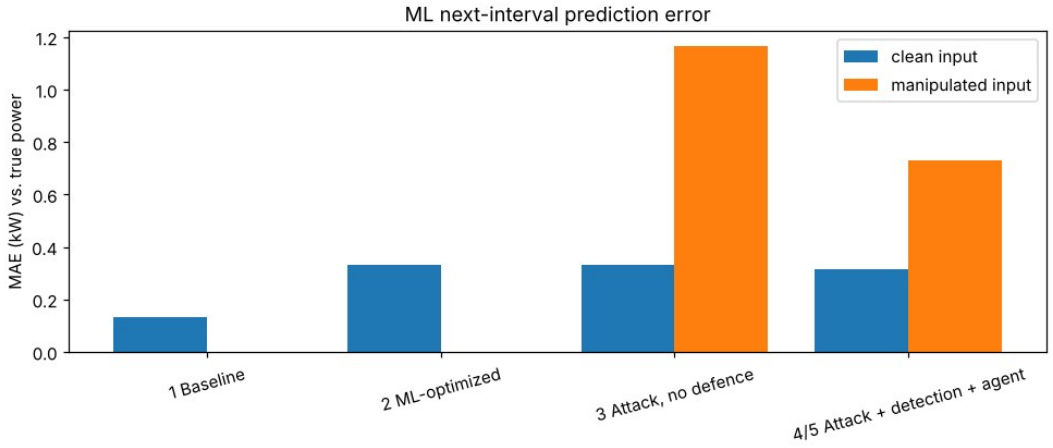


Figure 6.3. ML Prediction Accuracy per Scenario

Figure 6.3 shows prediction accuracy during the evaluation. The MAE is 0.131 kW under baseline control but 0.332 kW under optimiser control, because the model was trained on

baseline-controlled history. With attacks and no defence, manipulated inputs raised the error to 1.164 kW; isolating the manipulated sensors reduced it to 0.730 kW.

Table III. Attack Detection Performance (per Reading, after Warm-up)

Detectors	Precision	Recall	F1	FPR
Module 8 detectors (passive)	0.057	0.939	0.108	29.92 %
Retrained detectors (passive)	0.941	0.939	0.939	0.11 %
Retrained detectors (live, with agent)	0.318	0.939	0.468	4.35 %

Table III shows that detectors must match the control regime. Detectors trained on baseline-controlled data raised many false alarms under optimiser control (precision 0.057), whereas detectors retrained on optimiser-controlled data reached a precision of 0.941 and recall of 0.939. In the live, defended scenario precision dropped to 0.318, because zones that the agent returns to their baseline schedule again run under a different regime from the one the detectors were trained on.

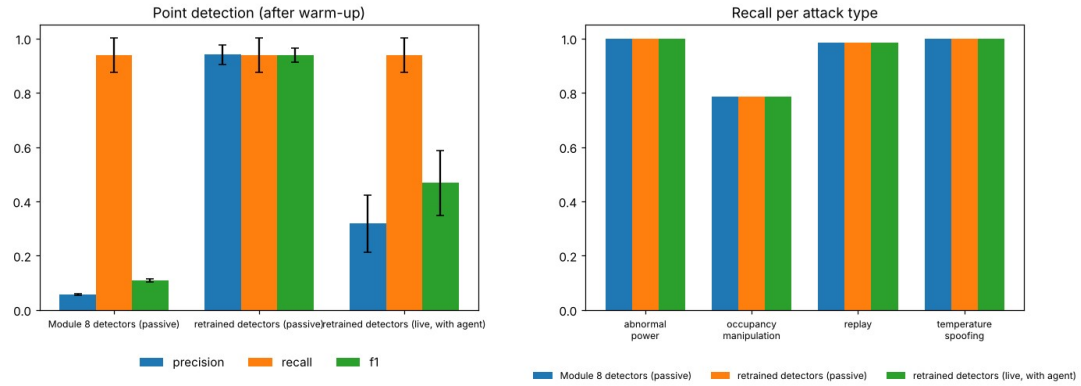


Figure 6.4. Detection Metrics and Detection Latency per Attack

Table IV. Containment and Recovery per Attack (Defended Scenario)

Attack	Zone	Detection (min)	Containment (min)	Recovery (min)
Temperature spoofing	Z1	0 ± 0	15 ± 0	155 ± 69
Occupancy manipulation	Z3	15 ± 9	33 ± 14	62 ± 40
Abnormal power	Z4	0 ± 0	15 ± 0	60 ± 0
Replay	Z2	2 ± 3	20 ± 0	1075 ± 69
Gradual manipulation	Z4	280 ± 125	170 ± 17	375 ± 0

As shown in Figure 6.4 and Table IV, all attacks were detected in all three seeds. Temperature spoofing, abnormal power and replay were detected within two simulated minutes on average and occupancy manipulation within 15 minutes; gradual manipulation, which is designed to stay below single-reading thresholds, was found by the drift tests after about four to five hours. Every attack was contained by sensor isolation and every quarantined sensor was restored after the zone had been quiet for an hour.

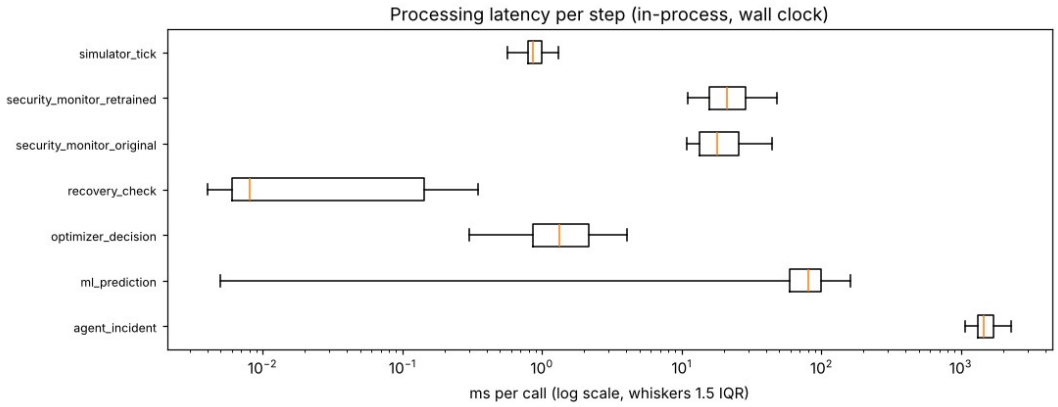


Figure 6.5. Processing Latency of the System Components

Table V. Kafka Transport Latency (Producer to Consumer)

Rate (msg/s)	Messages	Mean (ms)	p50 (ms)	p95 (ms)
5	1000	22.6	21.6	22.9
50	1000	23.8	21.2	23.6
500	1000	73.8	14.0	409.0

Figure 6.5 and Table V summarise processing latency. An ML prediction took 83.6 ms on average, an optimiser decision 2.1 ms, a security check 22–24 ms and a complete agent investigation 1.56 s. Kafka delivered messages in about 22 ms at the building's normal rate; at 500 messages per second the median stayed at 14 ms while the 95th percentile rose to 409 ms.

Dashboard and Operator Approval

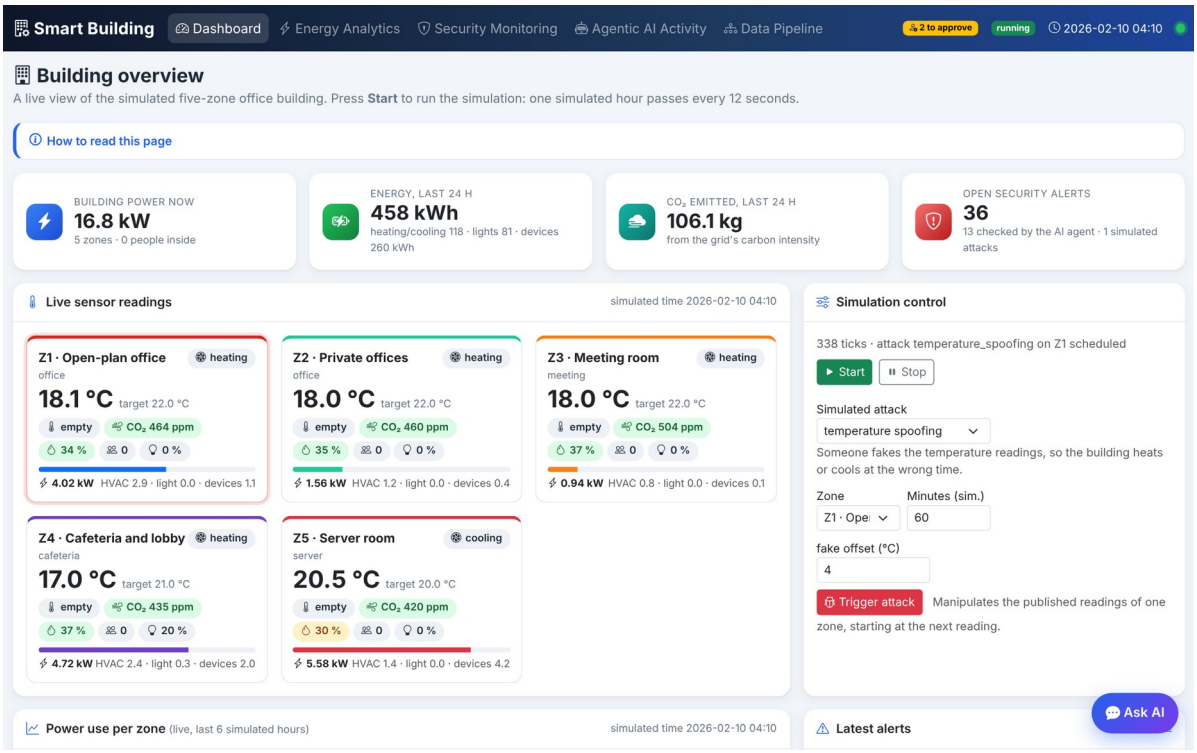


Figure 6.6. Live Building Overview Dashboard

Figure 6.6 shows the main dashboard with the building's power, energy and CO_2 of the last simulated day, open security alerts, live comfort-coded zone cards and the simulation controls used to start the simulation and launch simulated attacks.

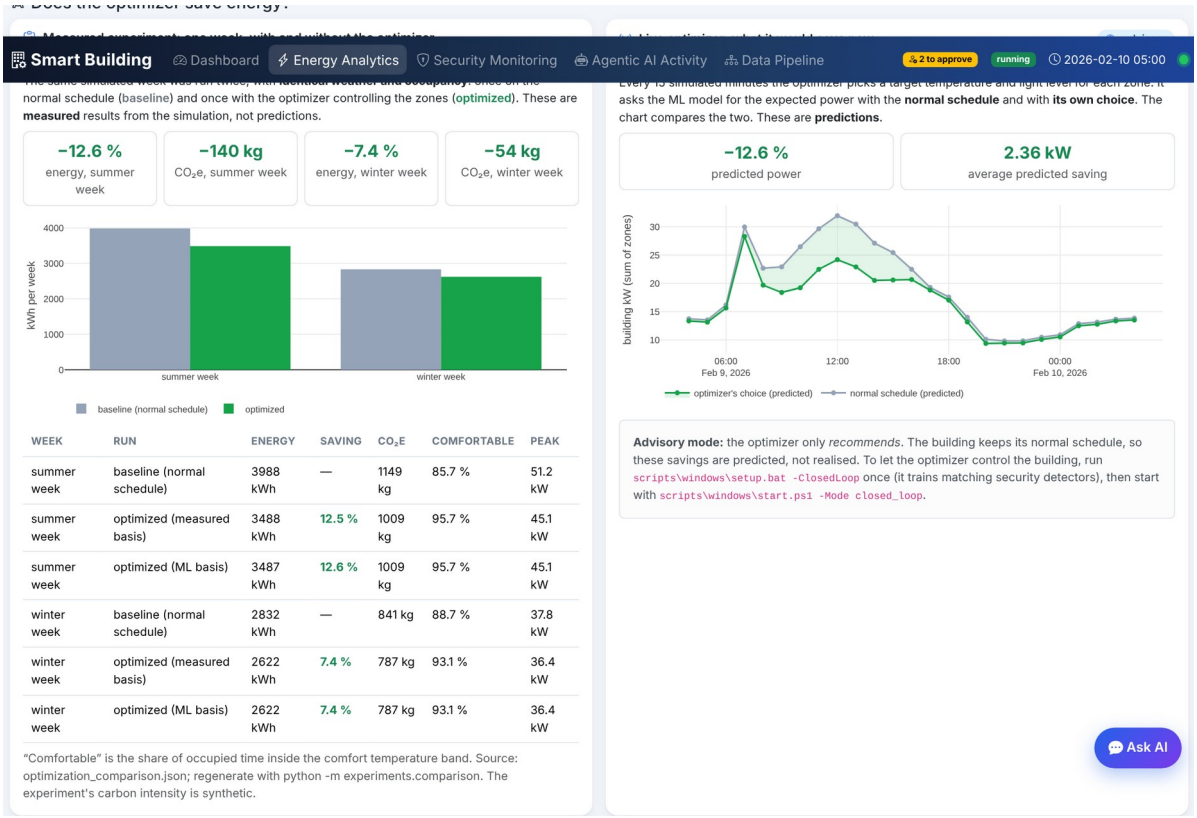


Figure 6.7. Measured and Live Comparison of Baseline and Optimized Operation

The Energy Analytics page in Figure 6.7 explains on what basis the optimiser is evaluated: the measured week-long experiment on the left and the optimiser's live predicted savings on the right.

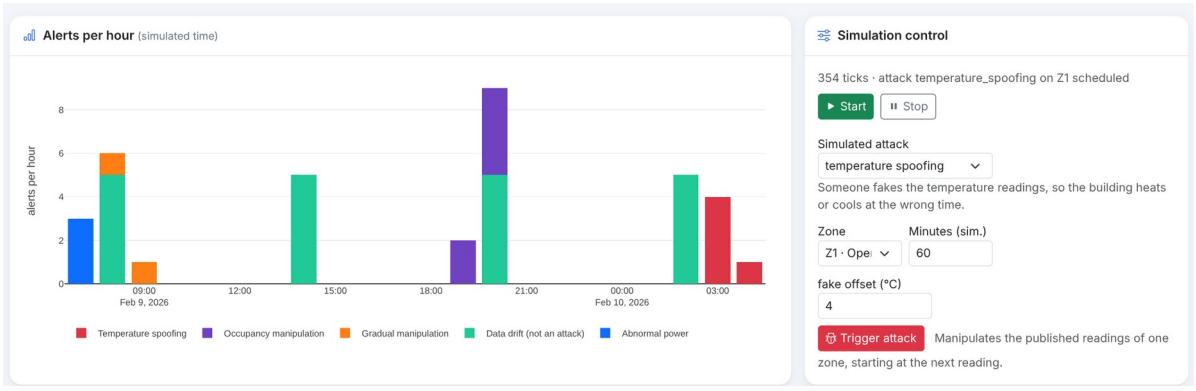


Figure 6.8. Security Monitoring: Alerts per Hour and Simulated Attack Controls

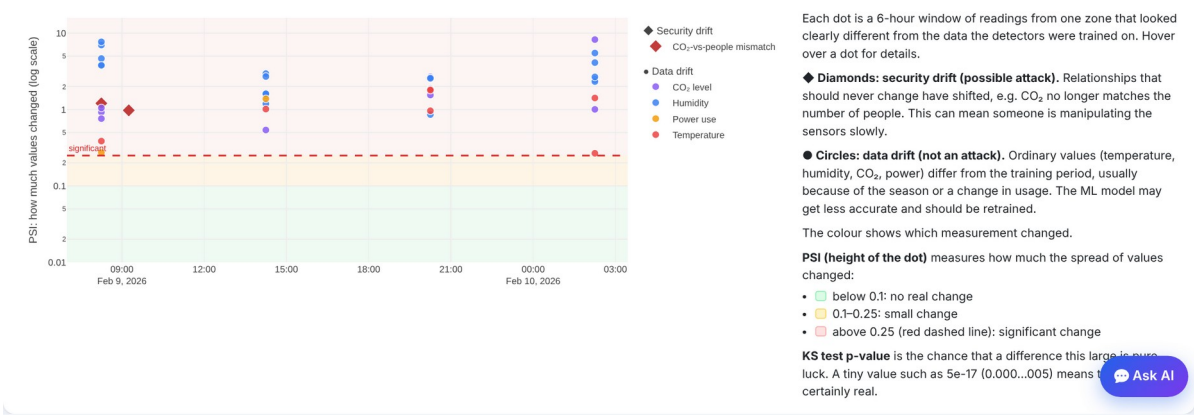


Figure 6.9. Drift Monitoring with PSI Bands and Explanation

Figure 6.8 shows the alerts per hour by type next to the controls used to launch an attack, and Figure 6.9 shows the drift chart, in which diamonds mark security drift and circles mark data drift, with coloured bands that explain the meaning of the PSI value.

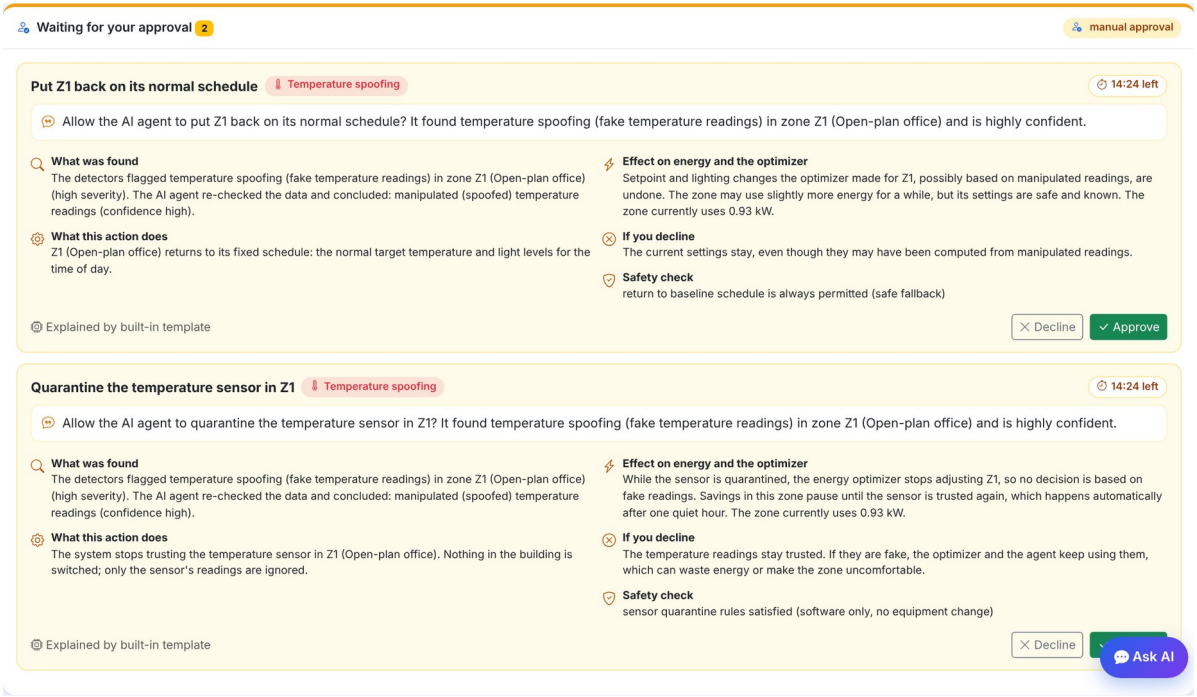


Figure 6.10. Agent Actions Waiting for Operator Approval with Plain-Language Explanations

Figure 6.10 shows the operator approval step after a temperature-spoofing attack on zone Z1. The agent proposed to quarantine the temperature sensor and to return the zone to its normal schedule; both proposals passed the safety validator and waited for the operator, each with an explanation of what was found, what the action does, its effect on energy and the optimiser, and what happens if it is declined. In the live test nothing was changed until the actions were approved; after approval both were executed and the sensor was quarantined.



Figure 6.11. Live Data Pipeline Visualisation

The Data Pipeline page in Figure 6.11 visualises the flow of data live: animated lines show where data is flowing in the last minute, and each box shows the throughput and totals of its stage and the number of messages retained per Kafka topic.

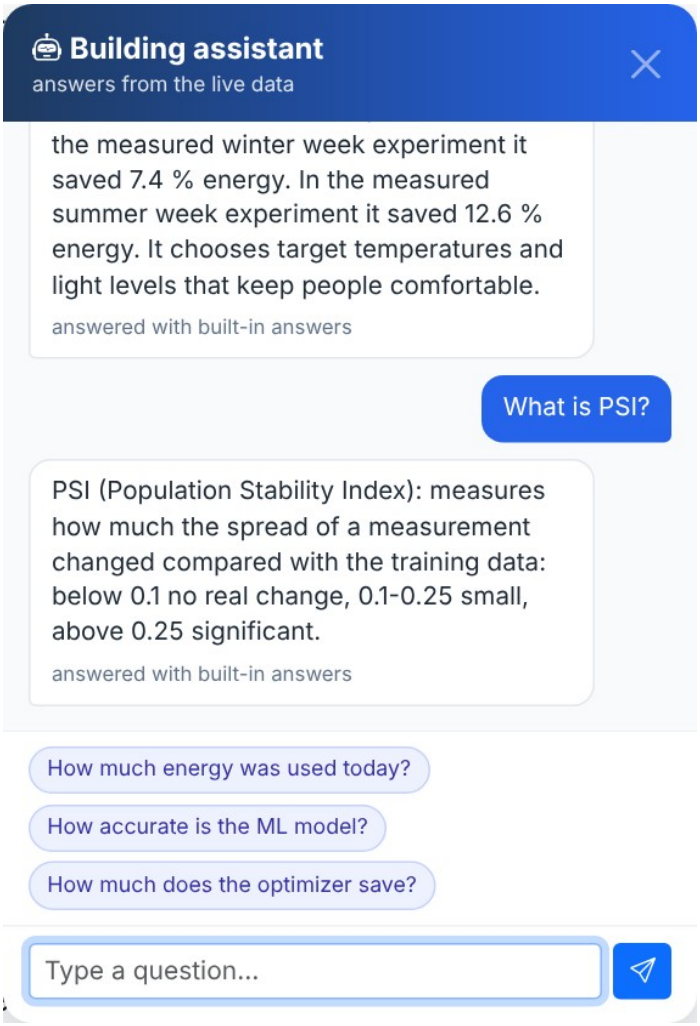


Figure 6.12. Chat Assistant Answering Questions from the Stored Data

Figure 6.12 shows the chat assistant available on every page. It answers questions about the building's data and models from facts stored in PostgreSQL and explains technical terms such as PSI.

Table VI. End-to-End System Verification (Both Control Modes)

Check	Advisory	Closed loop
Simulator → Kafka → workers → PostgreSQL	Pass	Pass
ML energy predictions	Pass	Pass
Energy optimisation	Pass	Pass
Cyberattack detection and live alert	Pass	Pass
Drift detection	Pass	Pass
Agent diagnosis and validated actions	Pass	Pass
Building feedback loop	Pass	Pass
Dashboard live updates	Pass	Pass
Sensor recovery	Pass (165 min)	Pass (625 min)
No service crashed (all 15 checks)	Pass	Pass

Table VI summarises the automated end-to-end verification, which starts the complete system in an isolated database and checks every stage. All checks passed in both control modes. In addition, all 434 automated tests passed.

6.2 DISCUSSION

The results show that a streaming architecture can support data-driven building optimisation in real time. The ML-based optimiser saved about 12 % of energy and 11 % of carbon emissions in the evaluation and between 7 % and 13 % in the seasonal comparison, while comfort improved rather than worsened, because the optimiser also corrects rooms that the fixed schedule leaves too cold or too warm.

The attack experiments demonstrate why protection is necessary: manipulated readings increased prediction errors almost fourfold. Detection was fast for sudden attacks and slower, but reliable, for gradual manipulation. The defended system contained every attack and retained 98 % of the savings after recovery. The cost of defence was a lower saving during the attack window, because quarantined zones fall back to the safe schedule, and slower recovery after a replay attack.

The most important limitation found is the dependence of the detectors on the control regime. Detectors trained on one regime raise false alarms under another, which is visible in the live precision of 0.318. Choosing detectors per zone according to its current control mode is a clear next step. The agent was evaluated with its deterministic offline policy for reproducibility; Claude and the optional small language model were integrated and tested with simulated interfaces but not evaluated live. The building, the attacks and the carbon profile of the evaluation are simulated.

Overall, the objectives of the project were achieved: the system streams, stores and processes data in real time, predicts and optimises energy use, detects and responds to cyberattacks with explainable, safety-checked and human-approved actions, and presents everything on a live dashboard.

CHAPTER-7
CONCLUSION

This chapter summarises the work carried out in the project and highlights its key outcomes. It reflects on how the developed system meets its objectives of real-time energy optimisation, cyberattack detection and safe, explainable response.

The project developed a complete, real-time smart building platform that combines streaming data processing, machine learning, optimisation, cybersecurity and agentic AI. A simulated five-zone building publishes its sensor readings to Apache Kafka, where independent services store and process them, compute energy and carbon emissions, predict power with XGBoost and choose comfortable, energy-efficient settings with a constrained optimiser.

The security layer detects sudden attacks with physics rules and an Isolation Forest and gradual manipulation with KS and PSI drift tests, and distinguishes attacks from ordinary data drift. An AI agent investigates alerts, explains its diagnosis and proposes corrective actions; a deterministic safety validator and, in manual mode, the operator's approval ensure that no unsafe or unwanted action is executed. Plain-language explanations, a chat assistant and a live pipeline view make the system understandable to non-experts.

In reproducible experiments, the optimiser reduced energy use by 12.2 % and carbon emissions by 10.9 % while improving comfort, all simulated attacks were detected and contained, and 98 % of the savings were retained after recovery. The system passed an automated end-to-end verification in both control modes and 434 automated tests, demonstrating that energy optimisation and its protection can be integrated in one reliable real-time application.

CHAPTER-8
FUTURE ENHANCEMENTS

This chapter discusses possible improvements and extensions to the system. It identifies opportunities to increase detection accuracy, adaptability, realism and scalability, and to make the agent and the operator interface more capable.

1. **Control-regime-aware detectors:** Select or retrain detectors per zone according to whether the zone is under optimiser or baseline control, to remove the false alarms observed after the agent returns a zone to its schedule.
2. **Continuous model retraining:** Retrain the energy model and detectors automatically when data drift is detected, including on optimiser-controlled history, to keep prediction errors low.
3. **Live LLM and SLM evaluation:** Evaluate the agent with Claude and the explanations and chat with a local small language model on real operator tasks, measuring diagnosis quality, cost and latency.
4. **Real building data and hardware:** Connect the pipeline to a real building management system or BACnet/MQTT sensors and validate savings on real buildings.
5. **Scalable deployment:** Run Kafka and the services on a multi-node cluster with replication and monitoring, so that many buildings can be served.
6. **Richer attack models and authentication:** Add network-level attacks, coordinated multi-zone attacks and message signing, and extend the operator interface with roles and audit reports.

APPENDIX

```
Energy_Optimization/
├── app.py                    Flask + Socket.IO dashboard
├── config.py                 settings from .env (validated)
├── simulator/                building physics, sensors, attacks
├── streaming/                Kafka producer, consumer, topics
├── workers/                  one process per service
│   ├── simulation_manager.py   sensor_persistence.py
│   ├── energy_processor.py     prediction_service.py
│   ├── optimization_service.py security_service.py
│   └── agent_service.py        record_persistence.py
├── processors/               validation, energy, features
├── database/                 schema.sql, repositories
├── ml/                       training, XGBoost models
├── optimization/             SLSQP optimizer, constraints
├── security/                 rules, Isolation Forest, drift
├── agent/                    LangGraph agent, tools, safety,
│                             approval gate, explainer, SLM
├── services/, routes/        dashboard data, chat, API
├── templates/, static/       pages, CSS, JavaScript
├── experiments/              evaluation and comparisons
├── scripts/                  bootstrap, preflight, run_stack,
│                             verify_system, windows/*.ps1
└── tests/                    434 automated tests
```

Figure 9.1. Project Structure

Figure 9.1 shows the structure of the project repository: the building simulator, the streaming services, the ML and optimisation modules, the security detectors, the agent, the dashboard and the experiments.

```
# one-time setup: virtual environment, packages, Docker, data and models
scripts\windows\setup.bat

# start Kafka, PostgreSQL, all services and the dashboard (http://127.0.0.1:5000)
scripts\windows\start.ps1
scripts\windows\start.ps1 -Mode closed_loop -MockExternal

# check the environment, verify the whole system end to end, run the tests
python -m scripts.preflight
python -m scripts.verify_system --launch --mode advisory
python -m pytest

# reproduce the evaluation and its tables and figures
python -m experiments.evaluation --out docs\results\evaluation
python -m experiments.evaluation_report --run docs\results\evaluation
```

Figure 9.2. Setup, Start-Up and Verification Commands (Windows)

Figure 9.2 lists the commands used to set up the system once, start all services with the dashboard, run the automated end-to-end verification and execute the test suite and the evaluation.

BIBLIOGRAPHY

[1] J. Kreps, N. Narkhede and J. Rao, “Kafka: A distributed messaging system for log processing,” in Proc. NetDB Workshop, Athens, Greece, 2011.

[2] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[3] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.

[4] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy and A. Bouchachia, “A survey on concept drift adaptation,” ACM Computing Surveys, vol. 46, no. 4, pp. 1–37, 2014.

[5] F. J. Massey Jr., “The Kolmogorov–Smirnov test for goodness of fit,” Journal of the American Statistical Association, vol. 46, no. 253, pp. 68–78, 1951.

[6] B. Yurdakul, “Statistical properties of population stability index,” Ph.D. dissertation, Western Michigan University, 2018.

[7] L. Pérez-Lombard, J. Ortiz and C. Pout, “A review on buildings energy consumption information,” Energy and Buildings, vol. 40, no. 3, pp. 394–398, 2008.

[8] K. Amasyali and N. M. El-Gohary, “A review of data-driven building energy consumption prediction studies,” Renewable and Sustainable Energy Reviews, vol. 81, pp. 1192–1205, 2018.

[9] J. Drgoňa et al., “All you need to know about model predictive control for buildings,” Annual Reviews in Control, vol. 50, pp. 190–232, 2020.

[10] Z. Wang and T. Hong, “Reinforcement learning for building controls: The opportunities and challenges,” Applied Energy, vol. 269, 115036, 2020.

[11] A. Humayed, J. Lin, F. Li and B. Luo, “Cyber-physical systems security—A survey,” IEEE Internet of Things Journal, vol. 4, no. 6, pp. 1802–1831, 2017.

[12] Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in Proc. Network and Distributed System Security Symp. (NDSS), 2018.

[13] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[14] L. M. Candanedo, V. Feldheim and D. Deramaix, “Data driven prediction models of energy use of appliances in a low-energy house,” Energy and Buildings, vol. 140, pp. 81–97, 2017.

[15] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” Nature Methods, vol. 17, pp. 261–272, 2020.

[16] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[17] LangChain, “LangGraph documentation.” [Online]. Available: https://langchain-ai.github.io/langgraph/

[18] Electricity Maps, “Electricity Maps API documentation.” [Online]. Available: https://portal.electricitymaps.com/docs

[19] Open-Meteo, “Free weather API documentation.” [Online]. Available: https://open-meteo.com/en/docs

[20] Ollama, “Run large language models locally.” [Online]. Available: https://ollama.com

[21] D. Kraft, “A software package for sequential quadratic programming,” DFVLR, Tech. Rep. DFVLR-FB 88-28, 1988.

[22] S. Amershi et al., “Guidelines for human-AI interaction,” in Proc. CHI Conf. Human Factors in Computing Systems, 2019, pp. 1–13.